

loan-approval-1

October 29, 2025

```
[3]: # Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[4]: #Step 2: Load Dataset
df = pd.read_csv('loan_sanction_test.csv')
df.head()
```

```
[4]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

	Credit_History	Property_Area
0	1.0	Urban
1	1.0	Urban
2	1.0	Urban
3	NaN	Urban
4	1.0	Urban

```
[5]: # Step 3: Basic Information
df.shape
```

```
[5]: (367, 12)
```

```
[6]: df.info()
      df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                367 non-null   object
1   Gender                 356 non-null   object
2   Married                367 non-null   object
3   Dependents             357 non-null   object
4   Education              367 non-null   object
5   Self_Employed          344 non-null   object
6   ApplicantIncome        367 non-null   int64
7   CoapplicantIncome      367 non-null   int64
8   LoanAmount             362 non-null   float64
9   Loan_Amount_Term       361 non-null   float64
10  Credit_History         338 non-null   float64
11  Property_Area          367 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

```
[6]:      ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
count      367.000000      367.000000    362.000000      361.000000
mean      4805.599455      1569.577657     136.132597      342.537396
std       4910.685399      2334.232099      61.366652       65.156643
min         0.000000         0.000000      28.000000        6.000000
25%       2864.000000         0.000000     100.250000     360.000000
50%       3786.000000      1025.000000     125.000000     360.000000
75%       5060.000000      2430.500000     158.000000     360.000000
max       72529.000000    24000.000000     550.000000     480.000000

      Credit_History
count      338.000000
mean         0.825444
std         0.380150
min         0.000000
25%         1.000000
50%         1.000000
75%         1.000000
max         1.000000
```

```
[7]: # Step 4: Check Missing Values
      df.isnull().sum()
      df.isnull().mean()*100
```

```
[7]: Loan_ID      0.000000
      Gender      2.997275
      Married     0.000000
      Dependents  2.724796
      Education   0.000000
      Self_Employed 6.267030
      ApplicantIncome 0.000000
      CoapplicantIncome 0.000000
      LoanAmount   1.362398
      Loan_Amount_Term 1.634877
      Credit_History 7.901907
      Property_Area 0.000000
      dtype: float64
```

```
[8]: # Step 5: Data Cleaning
df.fillna(df.mean(numeric_only=True), inplace=True) # fill with mean
df.fillna(df.median(numeric_only=True), inplace=True) # fill with median
```

```
[9]: df.dropna( inplace=True)
```

```
[10]: df.head()
```

```
[10]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

```
[11]: df.fillna(df.mode().iloc[0], inplace=True)
```

```
[12]: df.head()
```

```
[12]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

```
[13]: df.isnull().sum()    #Check again after filling
```

```
[13]: Loan_ID      0
      Gender      0
      Married     0
      Dependents  0
      Education   0
      Self_Employed 0
      ApplicantIncome 0
      CoapplicantIncome 0
      LoanAmount     0
      Loan_Amount_Term 0
      Credit_History 0
      Property_Area  0
      dtype: int64
```

```
[14]: df.duplicated().sum()
      df.drop_duplicates(inplace=True)
      df.head()
```

```
[14]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
0	5720	0	110.0	360.0
1	3076	1500	126.0	360.0
2	5000	1800	208.0	360.0
3	2340	2546	100.0	360.0
4	3276	0	78.0	360.0

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

```
[15]: df.rename(columns={'Loan_ID': 'LoanID', 'ApplicantIncome': 'Applicant_Income'},
    ↪ inplace=True)
```

```
[16]: df.head()
```

```
[16]:
```

	LoanID	Gender	Married	Dependents	Education	Self_Employed \
0	LP001015	Male	Yes	0	Graduate	No
1	LP001022	Male	Yes	1	Graduate	No
2	LP001031	Male	Yes	2	Graduate	No
3	LP001035	Male	Yes	2	Graduate	No
4	LP001051	Male	No	0	Not Graduate	No

	Applicant_Income	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
0	5720	0	110.0	360.0
1	3076	1500	126.0	360.0
2	5000	1800	208.0	360.0
3	2340	2546	100.0	360.0
4	3276	0	78.0	360.0

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

```
[29]: # Handle outliers using IQR
Q1 = df['LoanAmount'].quantile(0.25)
Q3 = df['LoanAmount'].quantile(0.75)
IQR = Q3 - Q1
df = df[~((df['LoanAmount'] < (Q1 - 1.5 * IQR)) | (df['LoanAmount'] > (Q3 + 1.5 *
    ↪ IQR)))]
```

```
df.head()
```

```
[29]:
```

	LoanID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	Applicant_Income	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

```
[30]: # Replace categorical text values with lowercase
df['Gender'] = df['Gender'].str.lower()
df['Married'] = df['Married'].str.lower()
```

```
[31]: df.head()
```

```
[31]:
```

	LoanID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	male	yes	0	Graduate	No	
1	LP001022	male	yes	1	Graduate	No	
2	LP001031	male	yes	2	Graduate	No	
3	LP001035	male	yes	2	Graduate	No	
4	LP001051	male	no	0	Not Graduate	No	

	Applicant_Income	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban

3	0.825444	Urban
4	1.000000	Urban

```
[32]: # Check data types and convert where necessary
df['Credit_History'] = df['Credit_History'].astype(int)
```

```
[33]: df.head()
```

```
[33]:
```

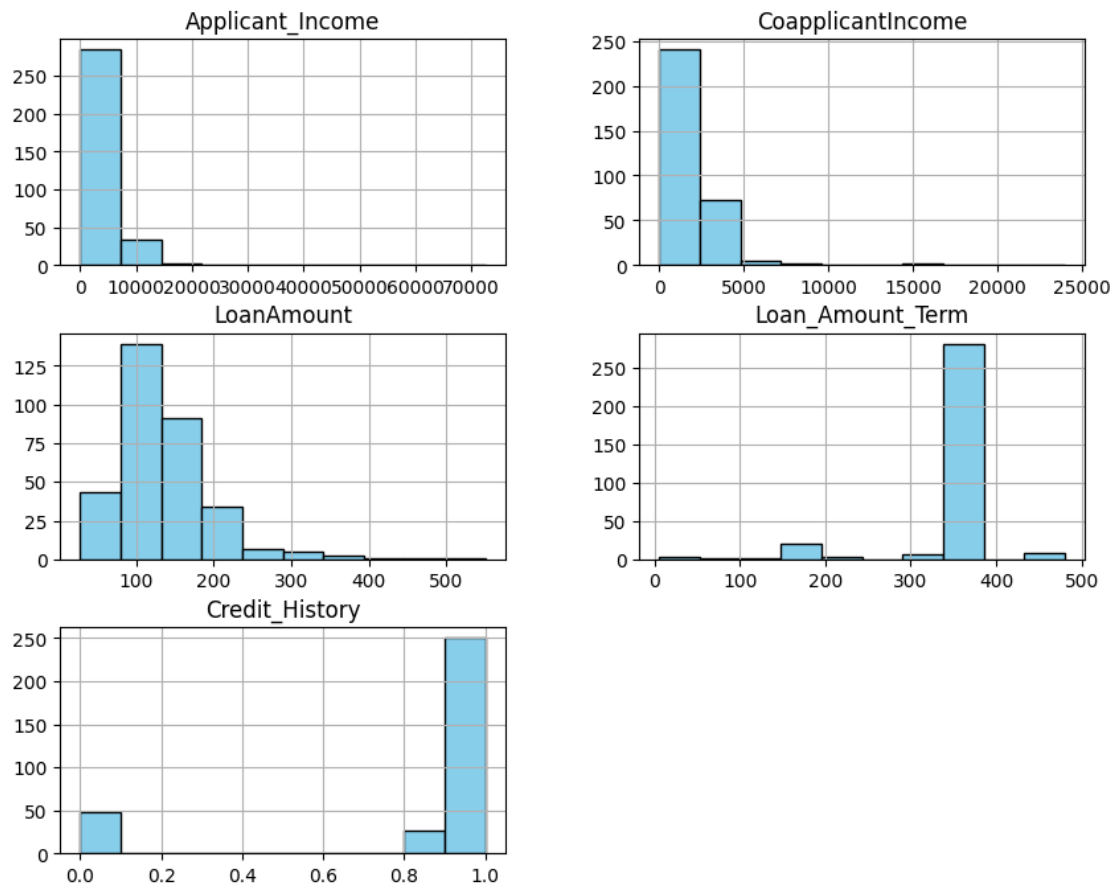
	LoanID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	male	yes	0	Graduate	No	
1	LP001022	male	yes	1	Graduate	No	
2	LP001031	male	yes	2	Graduate	No	
3	LP001035	male	yes	2	Graduate	No	
4	LP001051	male	no	0	Not Graduate	No	

	Applicant_Income	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	110.0	360.0	
1	3076	1500	126.0	360.0	
2	5000	1800	208.0	360.0	
3	2340	2546	100.0	360.0	
4	3276	0	78.0	360.0	

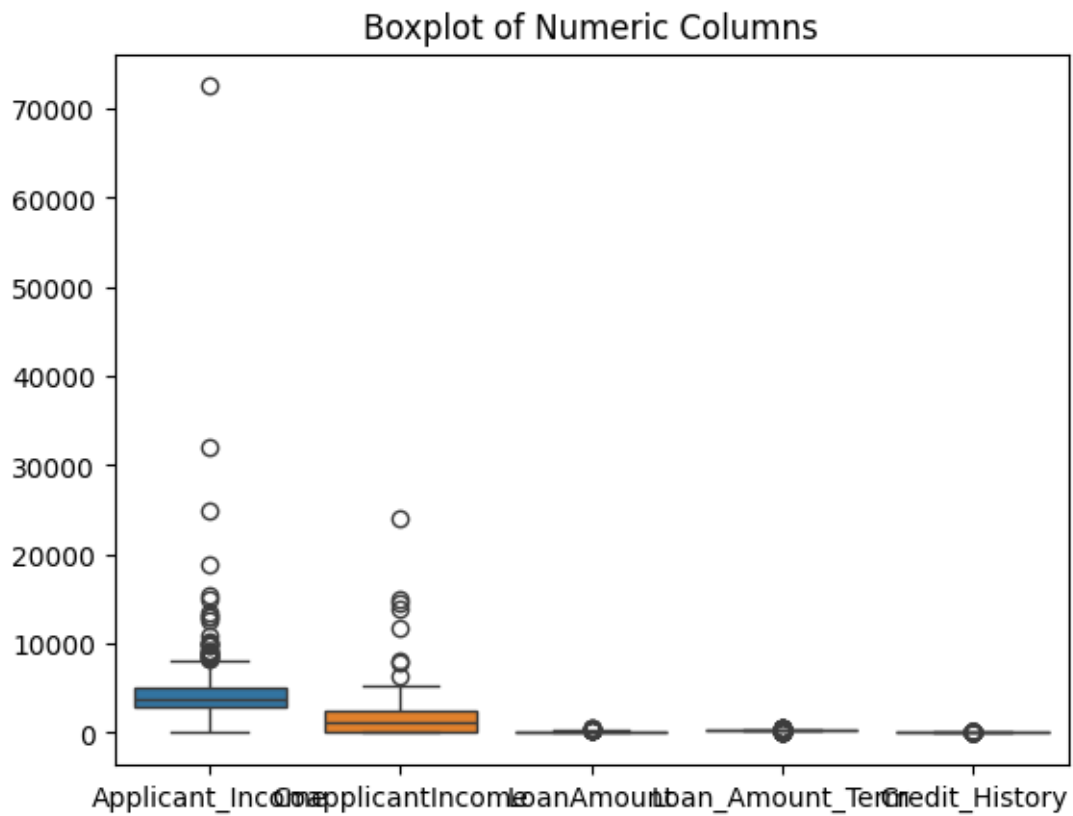
	Credit_History	Property_Area
0	1	Urban
1	1	Urban
2	1	Urban
3	0	Urban
4	1	Urban

```
[17]: # Step 6: Simple Visualizations
# Univariate Analysis
# Histogram
df.hist(figsize=(10,8), color='skyblue', edgecolor='black')
plt.suptitle('Numeric Columns Distribution')
plt.show()
```

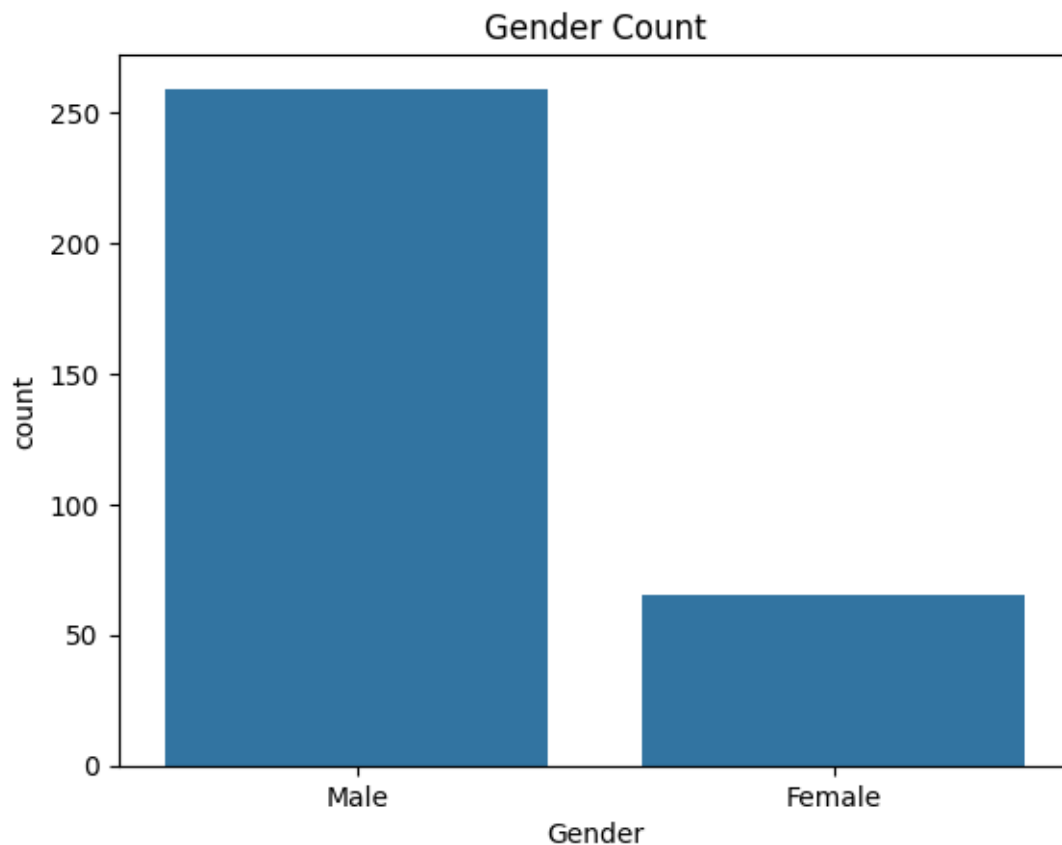
Numeric Columns Distribution



```
[18]: # Boxplot
sns.boxplot(data=df)
plt.title('Boxplot of Numeric Columns')
plt.show()
```



```
[19]: # Bar Chart
sns.countplot(x='Gender', data=df)
plt.title('Gender Count')
plt.show()
```

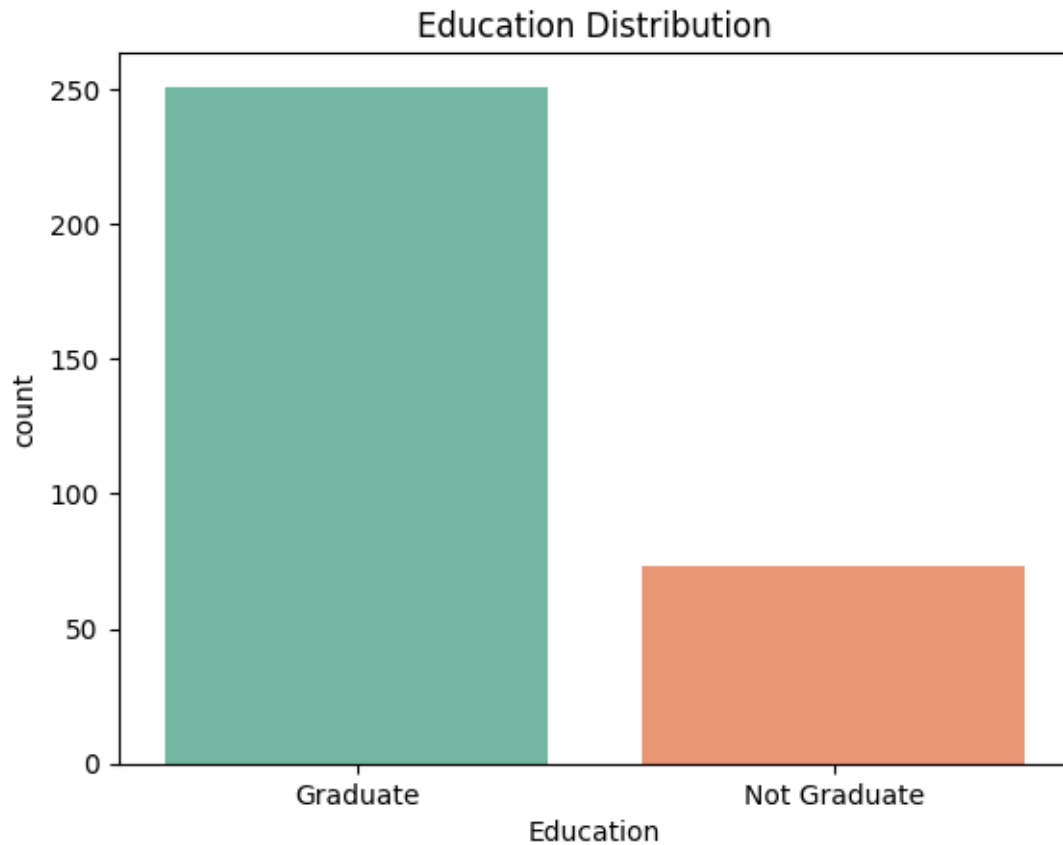


```
[20]: sns.countplot(x='Education', data=df, palette='Set2')
plt.title('Education Distribution')
plt.show()
```

/tmp/ipython-input-1676014983.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='Education', data=df, palette='Set2')
```

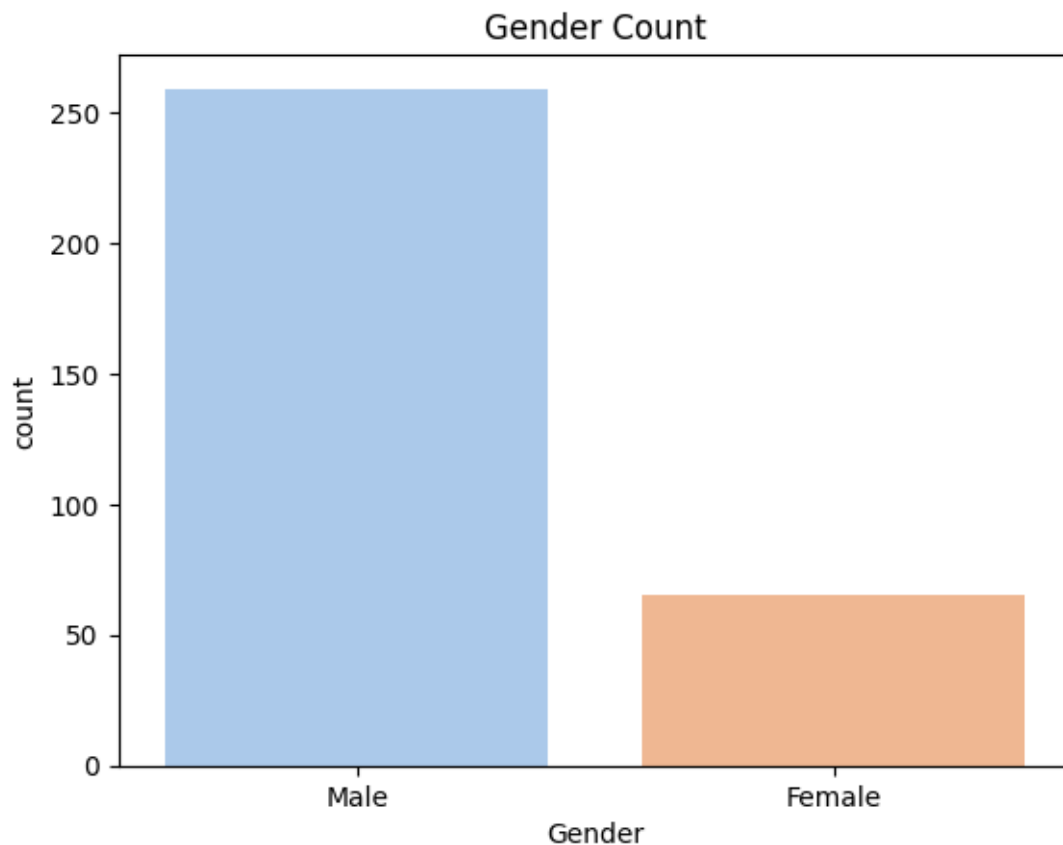


```
[21]: sns.countplot(x='Gender', data=df, palette='pastel')  
plt.title('Gender Count')  
plt.show()
```

/tmp/ipython-input-4150773090.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='Gender', data=df, palette='pastel')
```



```
[22]: sns.countplot(x='Property_Area', data=df, palette='coolwarm')  
plt.title('Property Area Distribution')  
plt.show()
```

/tmp/ipython-input-3236925448.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='Property_Area', data=df, palette='coolwarm')
```



```
[23]: df.columns
```

```
[23]: Index(['LoanID', 'Gender', 'Married', 'Dependents', 'Education',  
        'Self_Employed', 'Applicant_Income', 'CoapplicantIncome', 'LoanAmount',  
        'Loan_Amount_Term', 'Credit_History', 'Property_Area'],  
        dtype='object')
```

```
[24]: # Scatter Plot (Numeric Relationship)  
sns.scatterplot(x='Applicant_Income', y='LoanAmount', data=df, color='coral')  
plt.title('Applicant Income vs Loan Amount')  
plt.show()
```

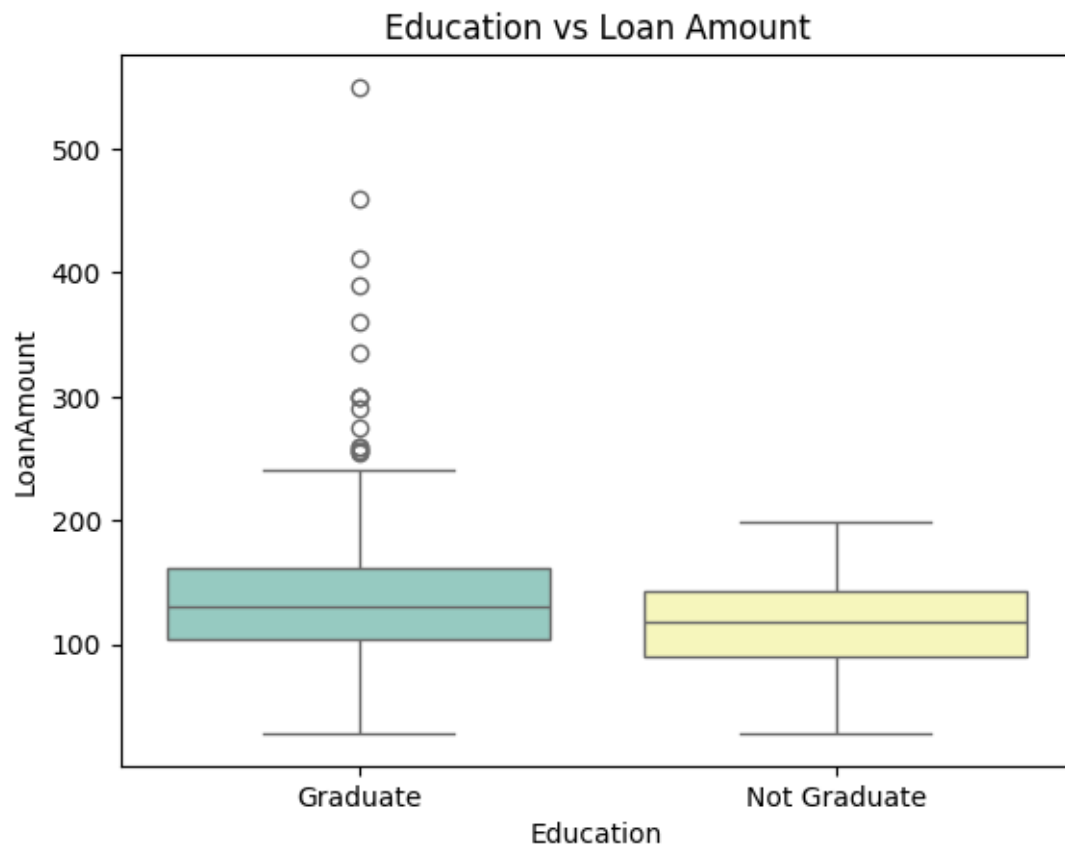


```
[25]: # Boxplot (Categorical vs Numeric)
sns.boxplot(x='Education', y='LoanAmount', data=df, palette='Set3')
plt.title('Education vs Loan Amount')
plt.show()
```

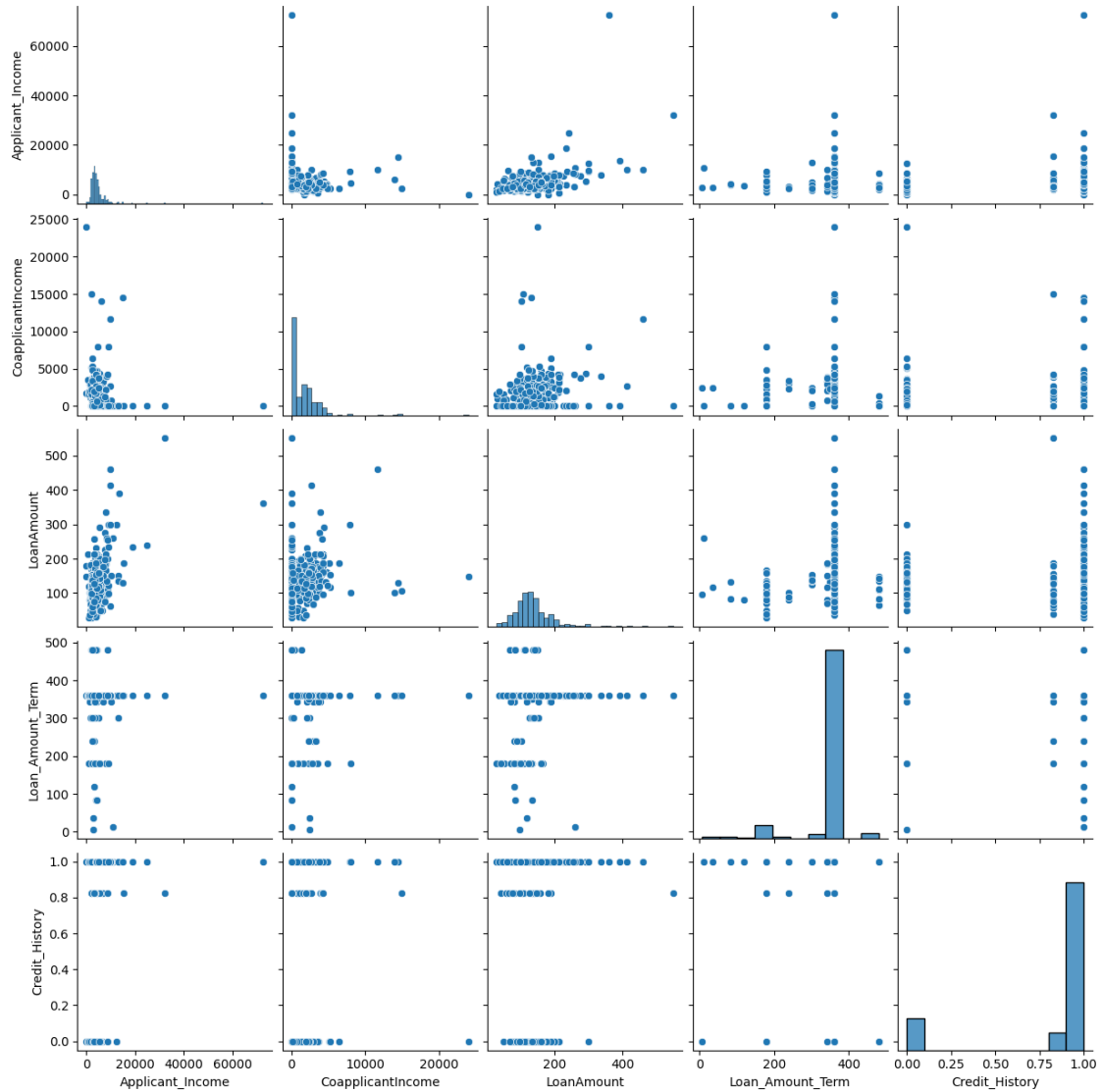
/tmp/ipython-input-1680359241.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

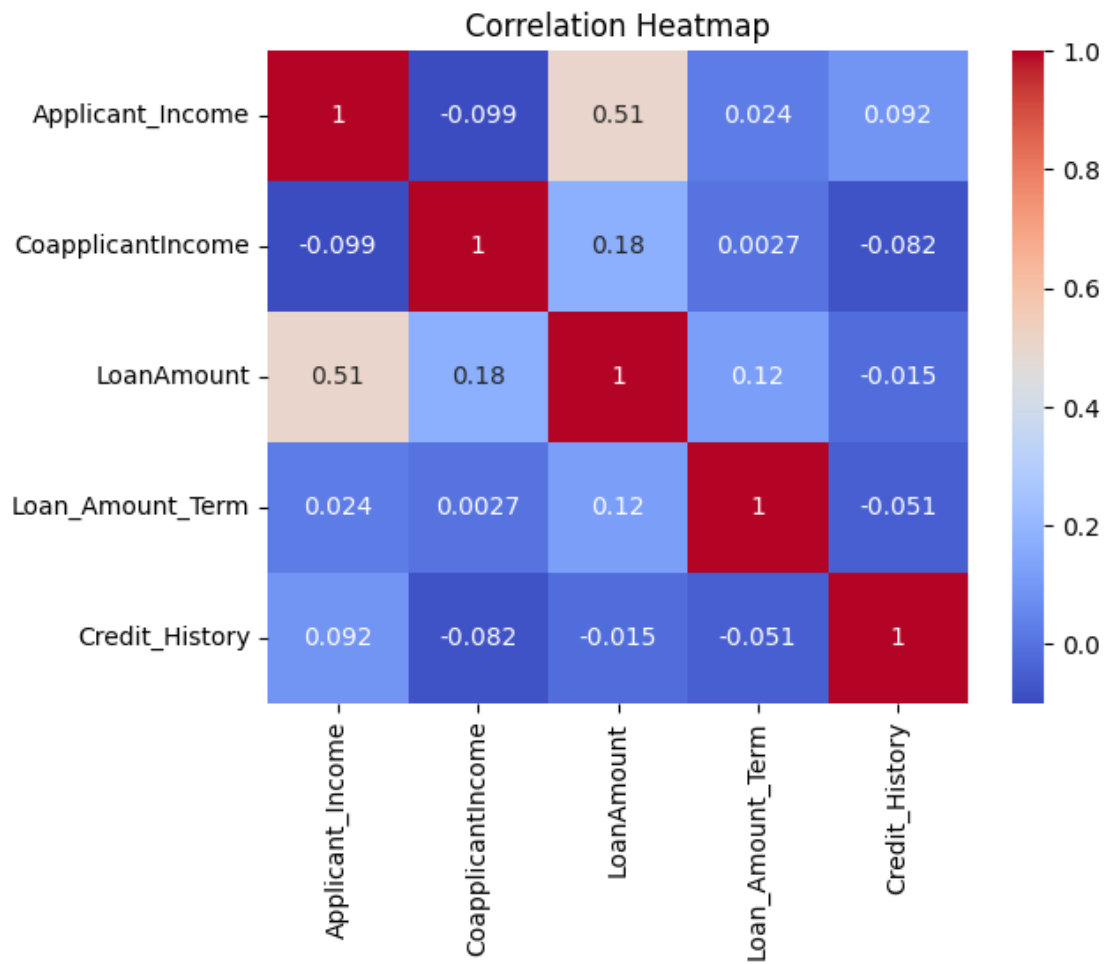
```
sns.boxplot(x='Education', y='LoanAmount', data=df, palette='Set3')
```



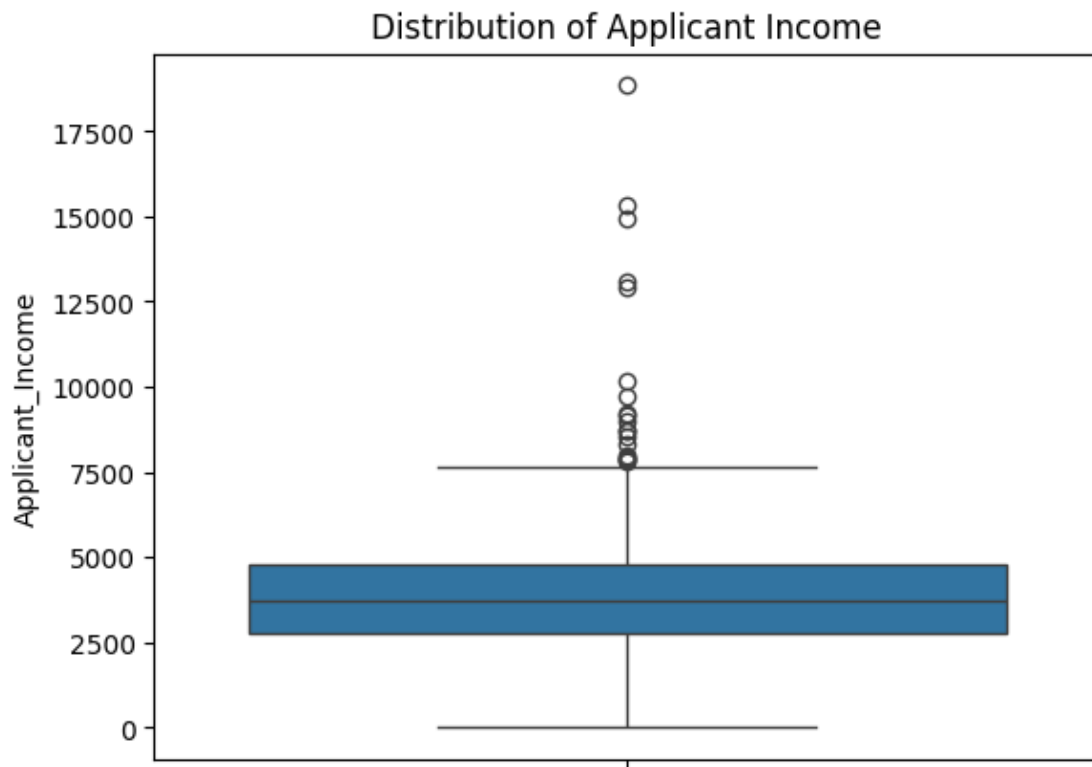
```
[26]: # Pair Plot (All Numeric Relationships)
sns.pairplot(df)
plt.show()
```



```
[27]: # Heatmap (Correlation)
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

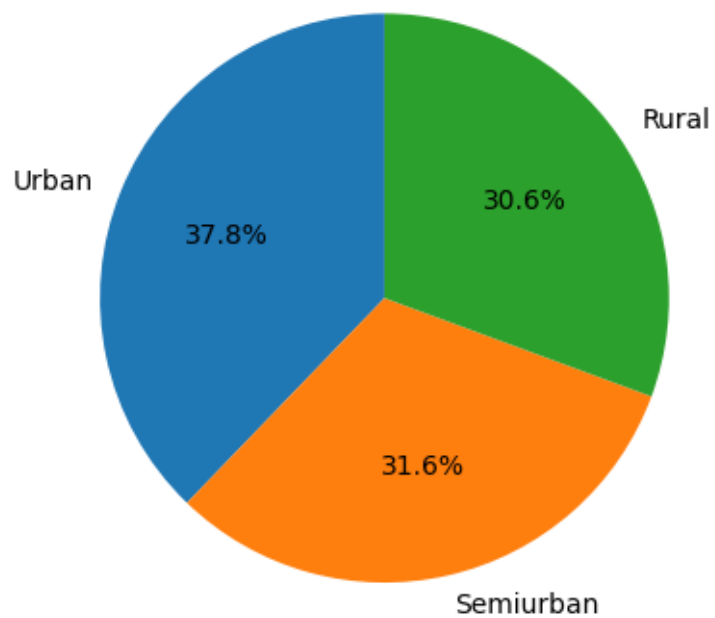


```
[34]: # Boxplot for Applicant Income
sns.boxplot(y='Applicant_Income', data=df)
plt.title('Distribution of Applicant Income')
plt.show()
```

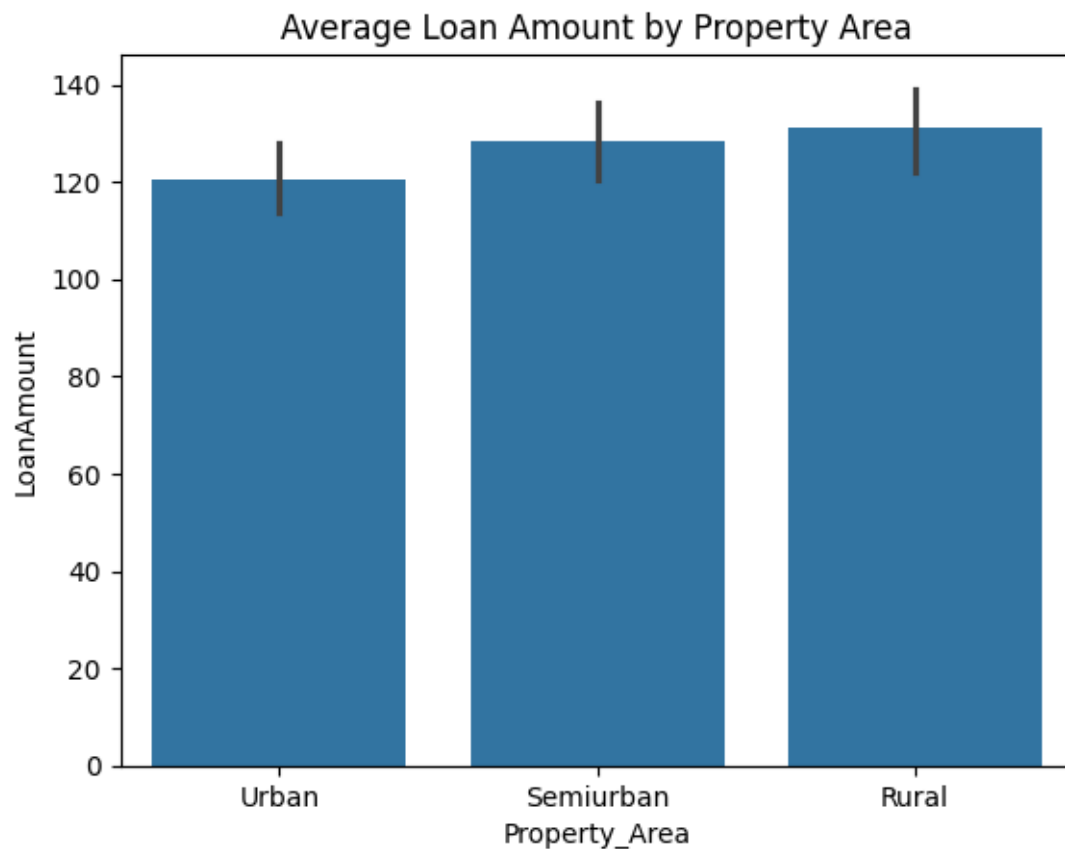


```
[35]: #Pie chart for Property Area distribution
df['Property_Area'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90)
plt.title('Property Area Distribution')
plt.ylabel('')
plt.show()
```

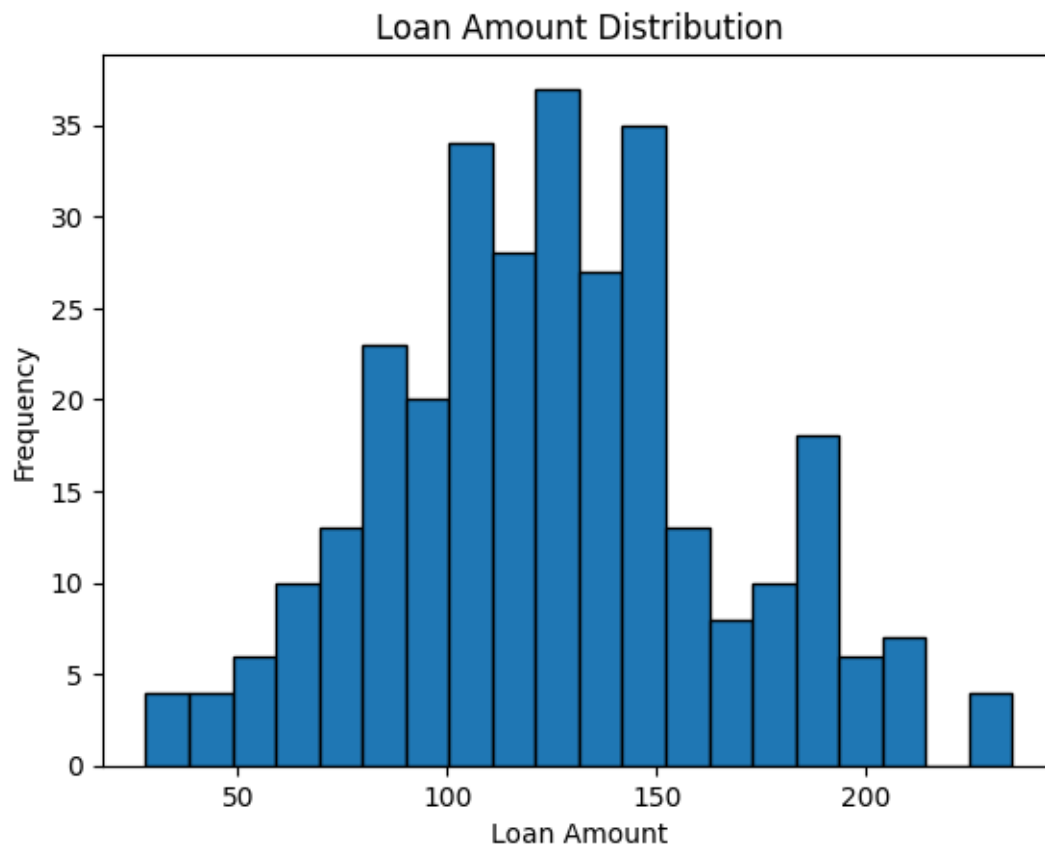
Property Area Distribution



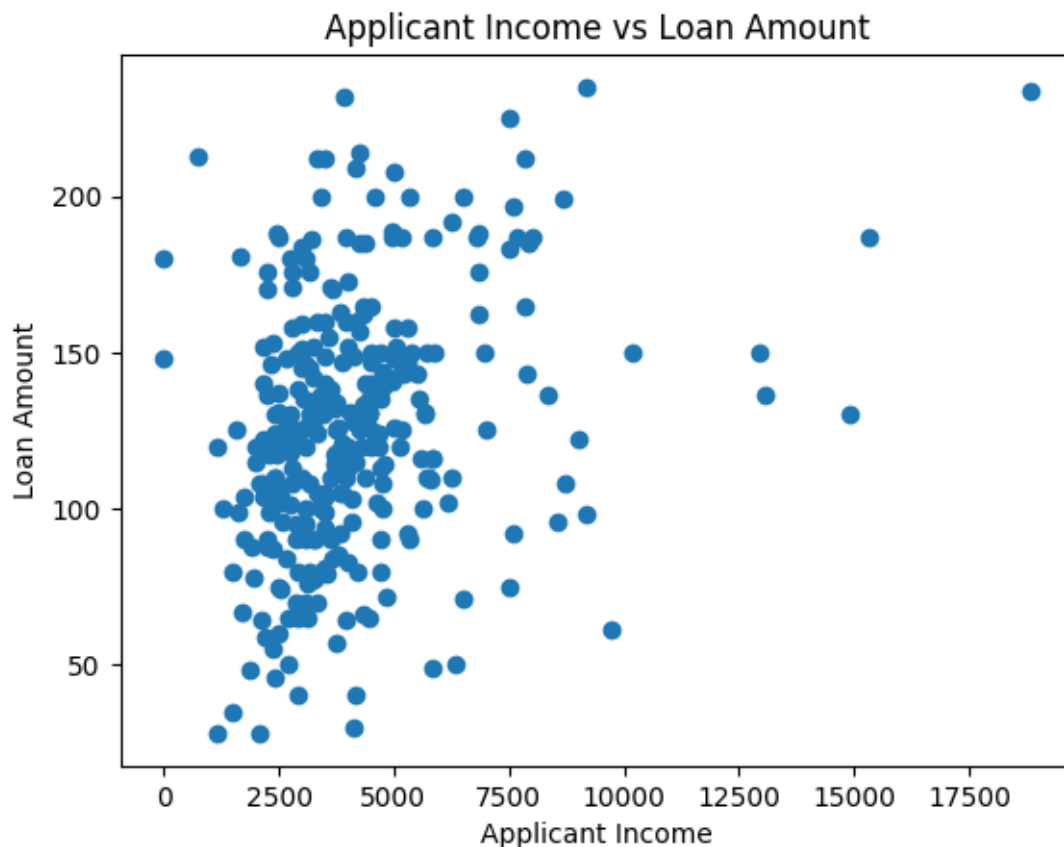
```
[36]: # Property Area vs Loan Amount
sns.barplot(x='Property_Area', y='LoanAmount', data=df)
plt.title('Average Loan Amount by Property Area')
plt.show()
```



```
[37]: # Histogram for Loan Amount
plt.hist(df['LoanAmount'], bins=20, edgecolor='black')
plt.title('Loan Amount Distribution')
plt.xlabel('Loan Amount')
plt.ylabel('Frequency')
plt.show()
```



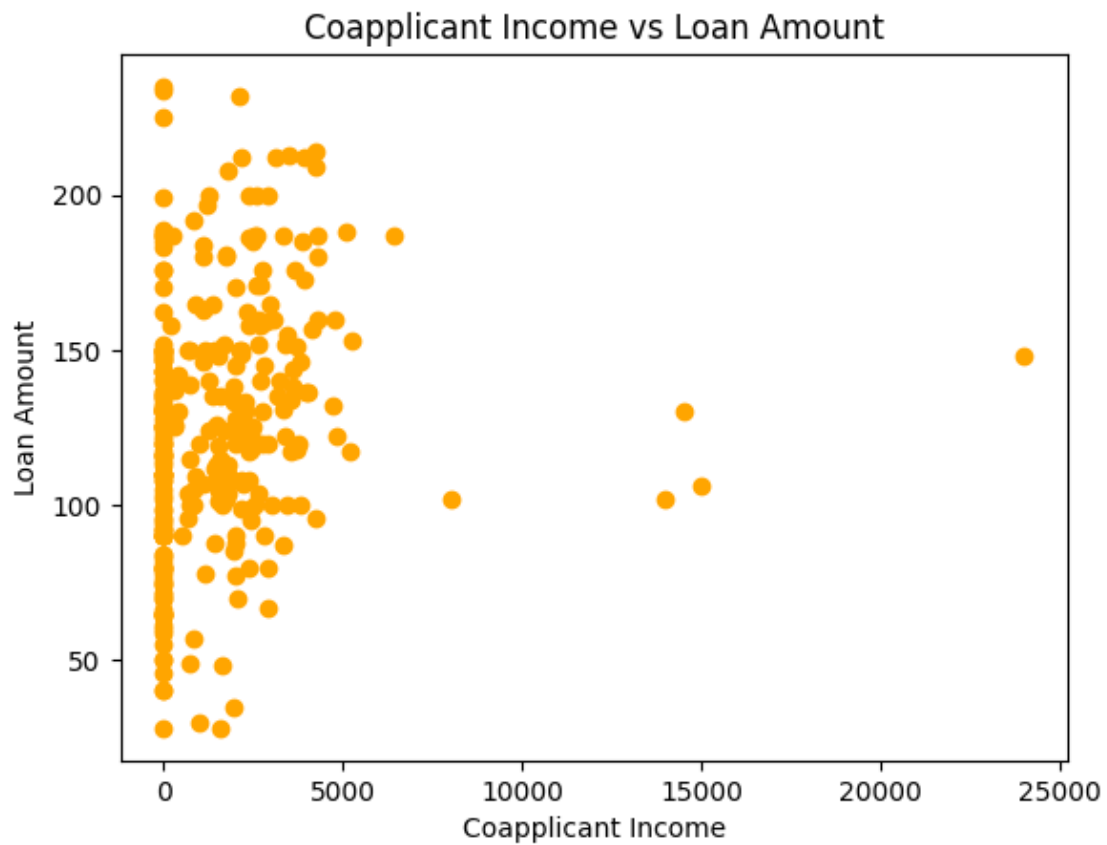
```
[38]: # Loan Amount vs Applicant Income (Scatter Plot)
plt.scatter(df['Applicant_Income'], df['LoanAmount'])
plt.title('Applicant Income vs Loan Amount')
plt.xlabel('Applicant Income')
plt.ylabel('Loan Amount')
plt.show()
```



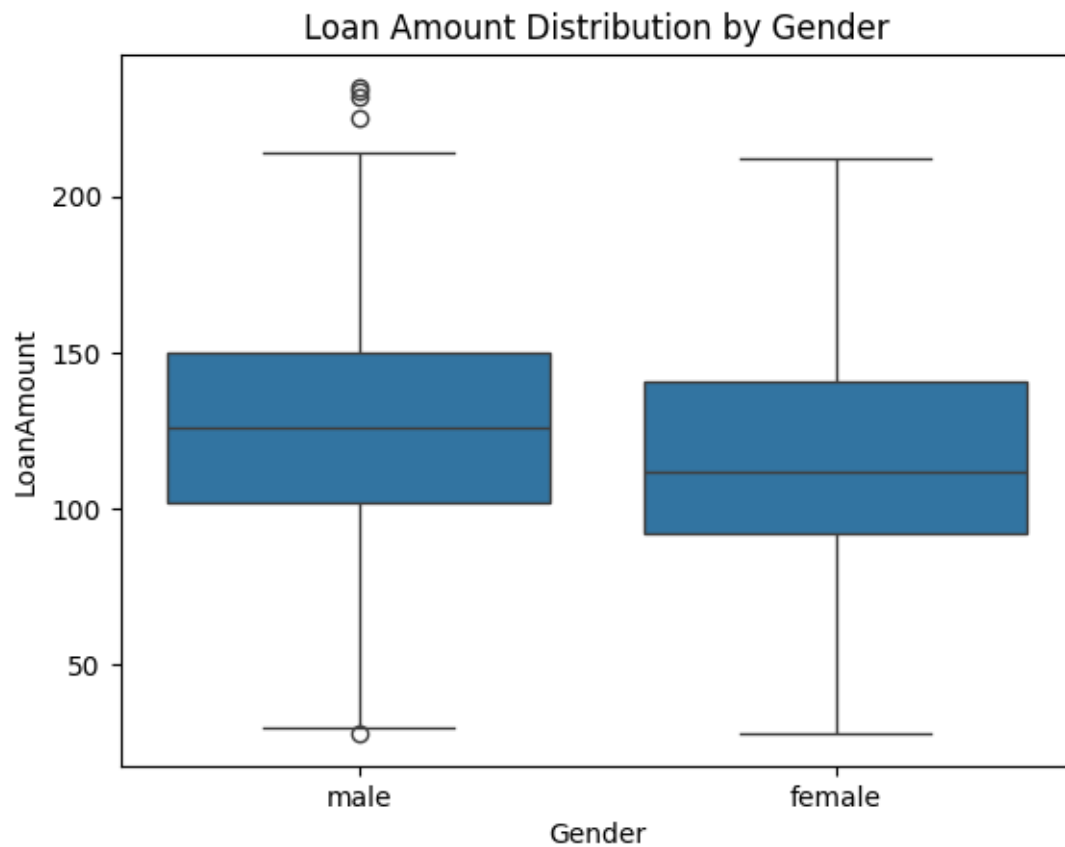
```
[40]: df.columns
```

```
[40]: Index(['LoanID', 'Gender', 'Married', 'Dependents', 'Education',
          'Self_Employed', 'Applicant_Income', 'CoapplicantIncome', 'LoanAmount',
          'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
         dtype='object')
```

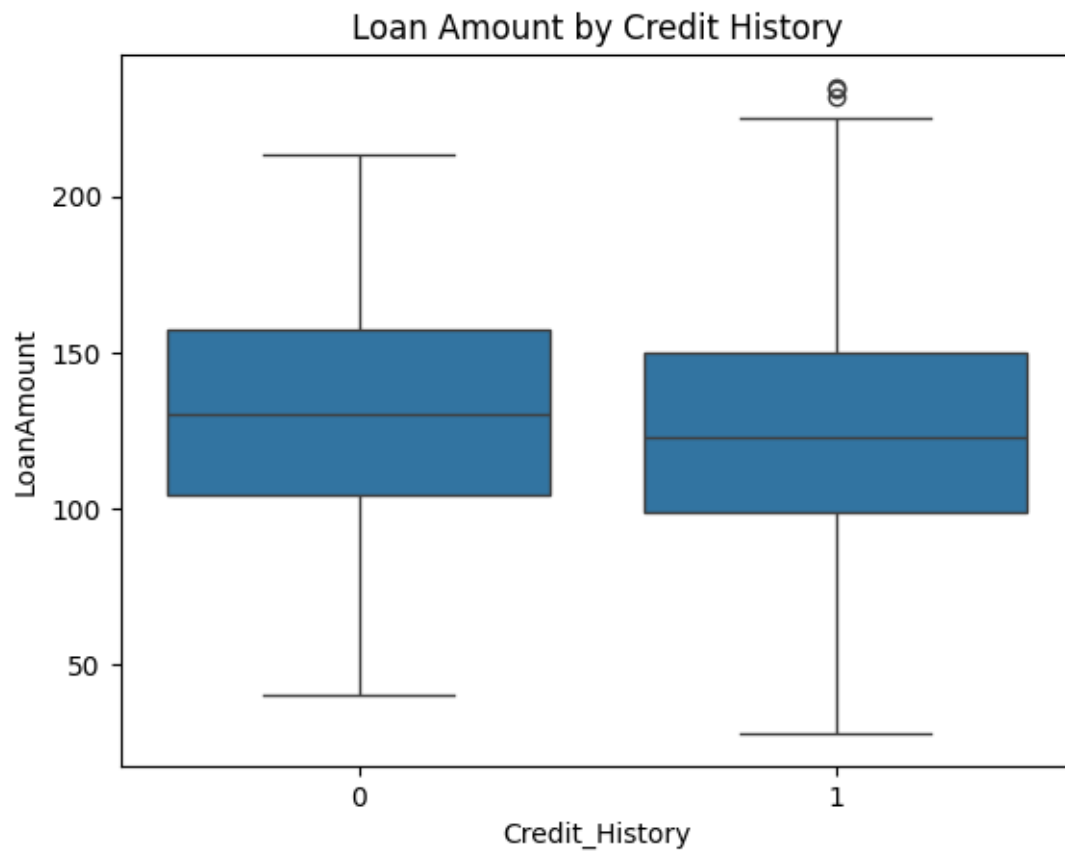
```
[41]: # Coapplicant Income vs Loan Amount
plt.scatter(df['CoapplicantIncome'], df['LoanAmount'], color='orange')
plt.title('Coapplicant Income vs Loan Amount')
plt.xlabel('Coapplicant Income')
plt.ylabel('Loan Amount')
plt.show()
```



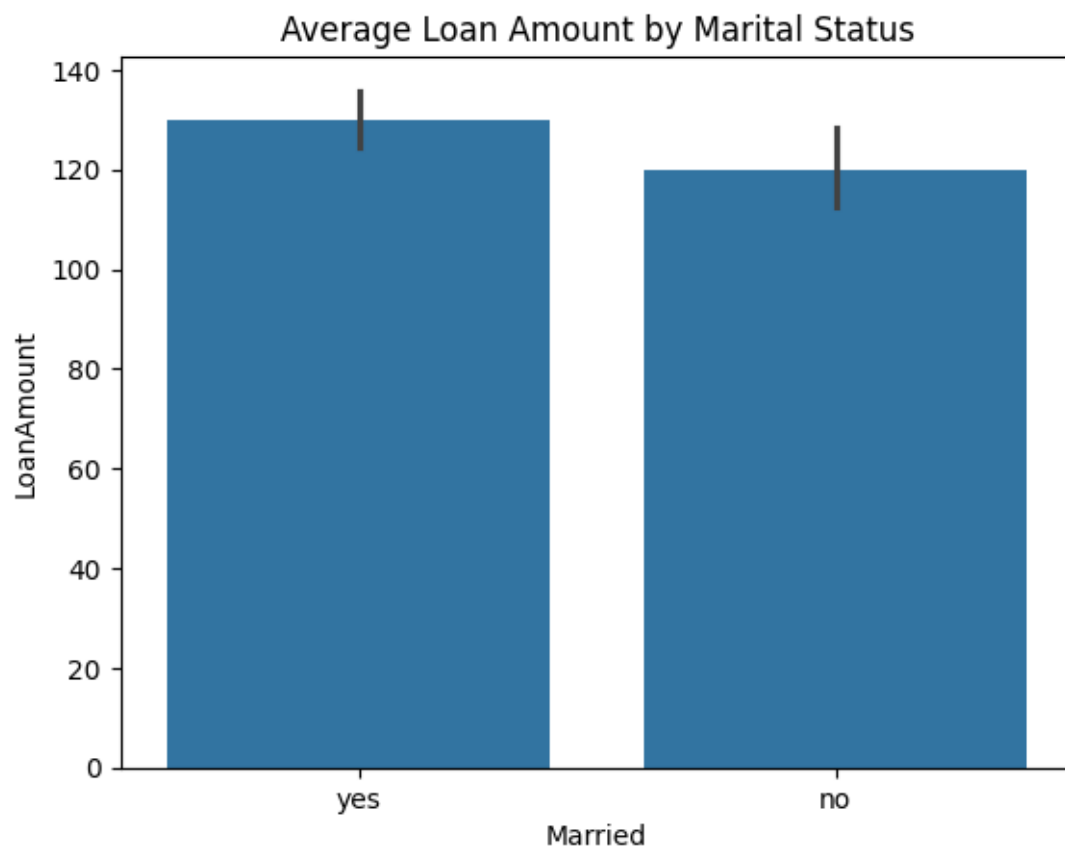
```
[43]: # Loan Amount distribution by Gender
sns.boxplot(x='Gender', y='LoanAmount', data=df)
plt.title('Loan Amount Distribution by Gender')
plt.show()
```



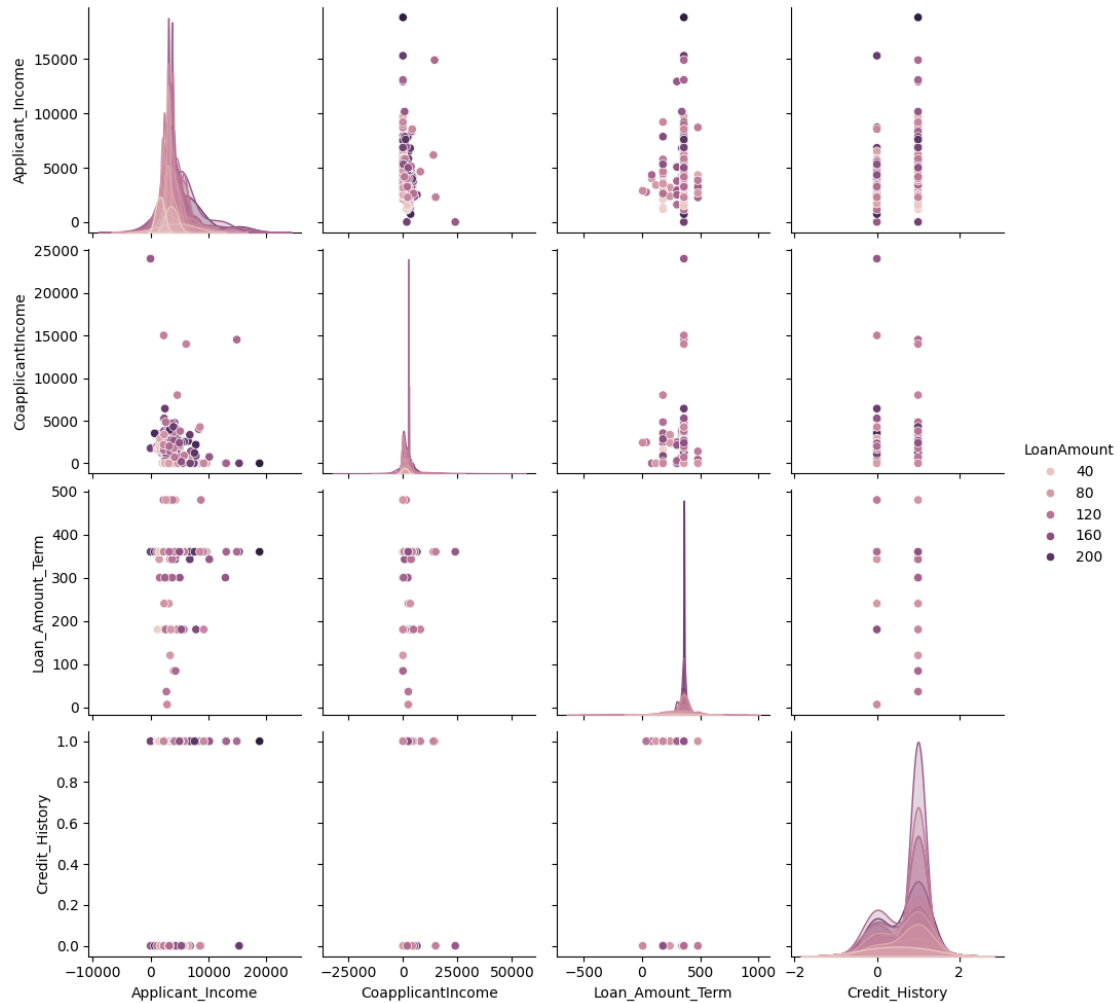
```
[44]: # Step 38: Loan Amount distribution by Credit History
sns.boxplot(x='Credit_History', y='LoanAmount', data=df)
plt.title('Loan Amount by Credit History')
plt.show()
```



```
[45]: # Average Loan Amount by Marital Status
sns.barplot(x='Married', y='LoanAmount', data=df, estimator=np.mean)
plt.title('Average Loan Amount by Marital Status')
plt.show()
```



```
[47]: # Pairplot showing correlation among numeric columns
sns.pairplot(df, hue='LoanAmount')
plt.show()
```



[28] : *# Simple Insight Notes*

- # 1. Most applicants are Male.*
- # 2. Urban area has more loan approvals.*
- # 3. Higher income → higher loan amount.*
- # 4. Graduates get more loans approved.*

[28] :